

Procesamiento y visualización de señales EEG

1. Introducción

Una vez finalizada la adquisición de datos del OpenBCI Cyton + Daisy, el programa realiza una serie de operaciones para preparar las señales para su visualización.

El procesamiento implementado actualmente comprende:

1. Conversión de los valores digitales de 24 bits.
2. Aplicación de un factor de escala para obtener valores expresados en microvoltios.
3. Construcción del eje temporal.
4. Aplicación de un desplazamiento vertical para separar visualmente los canales.
5. Generación de una gráfica con los 16 canales EEG.

El programa **no implementa actualmente filtrado digital, eliminación de artefactos ni análisis espectral.**

2. Datos obtenidos de la adquisición

Después de finalizar la adquisición, los datos se encuentran almacenados en:

`data_EEG`

La matriz tiene dimensiones:

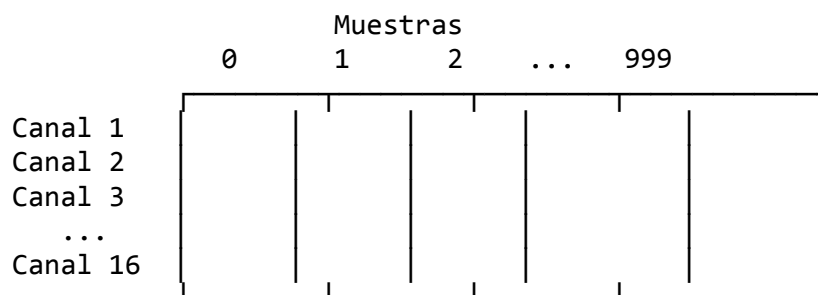
`(16, 1000)`

donde:

`16` → canales EEG

`1000` → muestras por canal

Conceptualmente:



Los valores almacenados en esta matriz son los valores digitales obtenidos a partir de las muestras de 24 bits de los paquetes recibidos.

3. Conversión de 24 bits

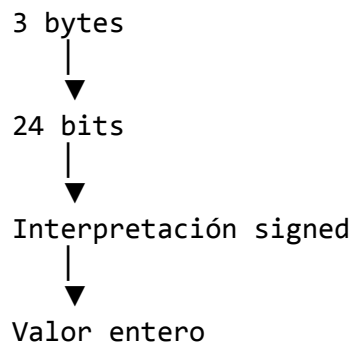
Cada muestra EEG recibida del OpenBCI está formada por tres bytes.

Estos tres bytes se combinan mediante la función:

```
def read_24bit_signed(b1, b2, b3):  
    value = (b1 << 16) | (b2 << 8) | b3  
  
    if value & 0x800000:  
        value -= 1 << 24  
  
    return value
```

El resultado de esta función es un entero con signo de 24 bits.

El proceso puede representarse como:



Este proceso permite convertir la representación binaria recibida mediante comunicación serial en un valor numérico que puede manipular Python.

4. Rango de los valores de 24 bits

Un entero signed de 24 bits puede representar:

-2^{23} hasta $2^{23} - 1$

Por lo tanto:

-8,388,608 hasta 8,388,607

El programa utiliza el bit más significativo para determinar si el valor debe interpretarse como positivo o negativo.

La máscara utilizada es:

`0x800000`

que corresponde al bit de signo del valor de 24 bits.

5. Conversión a microvoltios

Después de completar la adquisición y guardar el archivo CSV, el programa realiza una conversión de los valores digitales a una escala de microvoltios.

El factor utilizado es:

`scale = 4.5 / (24 * (2**23 - 1)) * 1e6`

Posteriormente se aplica:

`data_EEG_uV = data_EEG * scale`

El resultado se almacena en:

`data_EEG_uV`

La matriz mantiene las mismas dimensiones:

`(16, 1000)`

La diferencia es que ahora los valores están escalados para ser representados en microvoltios.

6. Descomposición del factor de escala

La expresión utilizada es:

`4.5 / (24 * (2**23 - 1)) * 1e6`

Puede separarse conceptualmente en:

4.5
|
├── valor utilizado como referencia de escala
└── divisor asociado con la resolución del ADC

y:

$$2^{23} - 1$$

representa el máximo valor positivo de un entero signed de 24 bits.

Finalmente:

1e6

realiza la conversión de voltios a microvoltios:

$$1 \text{ V} = 1,000,000 \text{ } \mu\text{V}$$

Nota

La fórmula utilizada forma parte de la implementación actual del proyecto. Para documentación experimental o publicación de resultados, el factor de conversión debe mantenerse consistente con la configuración de ganancia y referencia analógica utilizada por el hardware.

7. Datos originales y datos escalados

El programa mantiene dos matrices conceptualmente diferentes:

```
data_EEG
└── Valores digitales obtenidos del ADC

data_EEG_uV
└── Valores después de aplicar el factor de escala
```

La operación:

$$\text{data_EEG_uV} = \text{data_EEG} * \text{scale}$$

no modifica data_EEG.

Esto permite conservar los valores originales y utilizar una segunda matriz para la representación escalada.

8. Construcción del eje temporal

Para representar las señales como función del tiempo se crea:

```
t = np.arange(N_points) / fs
```

Los parámetros utilizados son:

```
N_points = 1000  
fs = 250
```

`np.arange(N_points)` genera:

0, 1, 2, 3, ..., 999

Posteriormente cada posición se divide entre `fs`.

Conceptualmente:

índice de muestra
↓
/ fs
↓
tiempo correspondiente

Por ejemplo:

```
Muestra 0 → 0/fs  
Muestra 1 → 1/fs  
Muestra 2 → 2/fs  
...  
Muestra 999 → 999/fs
```

9. Función de `fs`

En este programa:

```
fs = 250
```

se utiliza para construir el eje temporal:

```
t = np.arange(N_points) / fs
```

Es importante distinguir entre:

frecuencia de muestreo física

y:

valor utilizado por Python para interpretar el tiempo

Modificar únicamente:

`fs = 250`

a otro valor **no modifica la frecuencia de adquisición del OpenBCI**.

El valor utilizado en el código debe corresponder a la frecuencia real con la que fueron adquiridas las muestras si se desea que el eje temporal represente correctamente el tiempo.

10. Desplazamiento vertical de las señales

Cuando se grafican simultáneamente 16 señales, todas podrían aparecer superpuestas.

Para facilitar la visualización se utiliza:

`offset = 100`

Después cada canal se grafica mediante:

```
plt.plot(t, data_EEG_uV[ch] + ch*offset)
```

El término:

`ch * offset`

genera un desplazamiento diferente para cada canal.

Por ejemplo:

Canal 1 → 0×100

Canal 2 → 1×100

Canal 3 → 2×100

Canal 4 → 3×100

...

Canal 16 → 15×100

El desplazamiento únicamente modifica la posición visual de la señal.

11. El desplazamiento no modifica los datos

Es importante distinguir:

`data_EEG_uV[ch]`

de:

`data_EEG_uV[ch] + ch*offset`

El primero representa los datos escalados del canal.

El segundo se utiliza únicamente como entrada para la gráfica.

Por lo tanto, el desplazamiento:

```
ch * offset
```

no modifica:

```
data_EEG
```

ni:

```
data_EEG_uV
```

12. Generación de la gráfica

La figura se crea mediante:

```
plt.figure(figsize=(12,10))
```

Esto establece un tamaño de:

```
12 x 10
```

para la figura.

Posteriormente se recorren los 16 canales:

```
for ch in range(16):  
    plt.plot(t, data_EEG_uV[ch] + ch*offset)
```

Cada iteración genera una curva correspondiente a un canal.

13. Configuración de la gráfica

El título se establece mediante:

```
plt.title("EEG 16 canales - OpenBCI Cyton + Daisy")
```

El eje horizontal representa el tiempo:

```
plt.xlabel("Tiempo (s)")
```

Los valores del eje vertical se ocultan:

```
plt.yticks([])
```

Esto permite concentrar la visualización en la forma de las señales sin mostrar las marcas del eje vertical.

Finalmente:

```
plt.tight_layout()
```

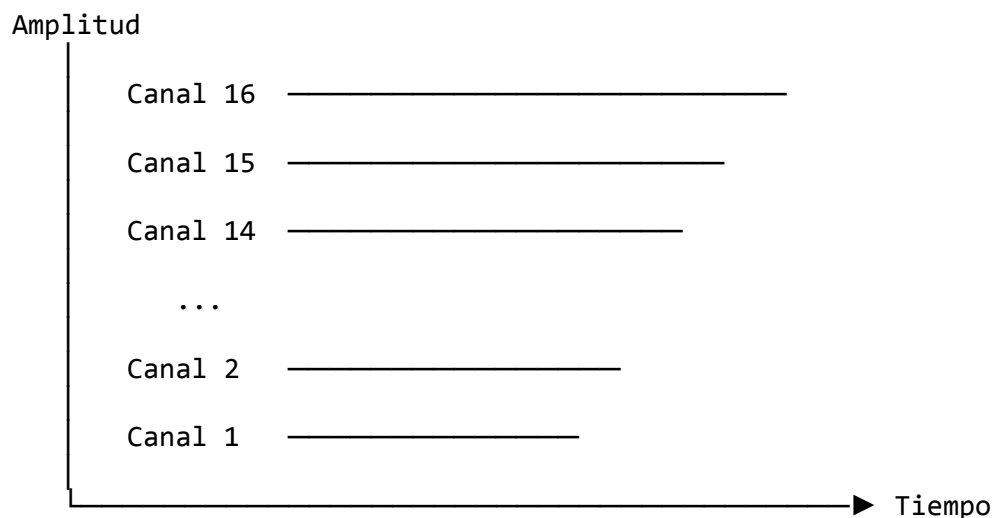
ajusta automáticamente la distribución de los elementos de la figura.

La gráfica se muestra mediante:

```
plt.show()
```

14. Estructura visual de la gráfica

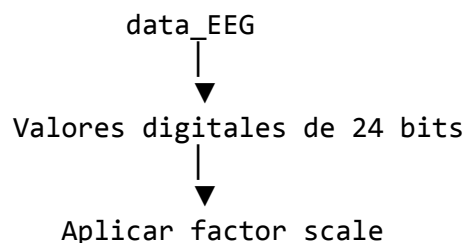
El resultado esperado tiene una estructura similar a:

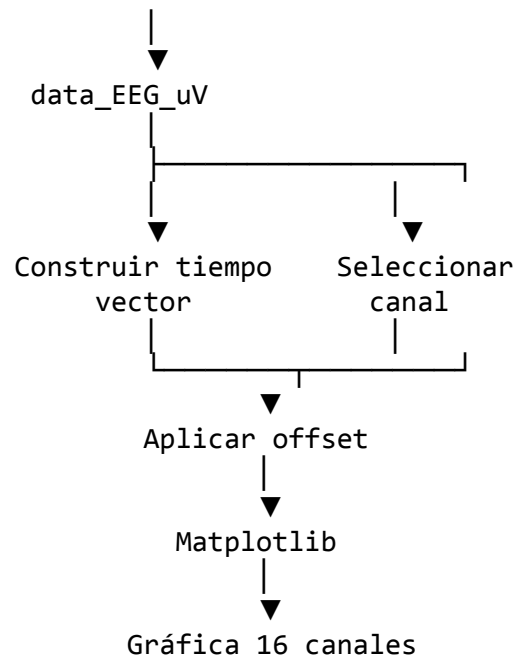


Las señales aparecen desplazadas verticalmente para facilitar su inspección.

15. Flujo del procesamiento

La etapa posterior a la adquisición puede resumirse como:





16. Procesamiento que actualmente NO realiza el programa

El código actual está orientado principalmente a:

- adquisición;
- conversión;
- almacenamiento;
- visualización.

No se implementan actualmente:

Filtrado

No existe un filtro digital pasa-banda, pasa-bajas o pasa-altas.

Filtro Notch

No existe actualmente un filtro específico para eliminar interferencia de la red eléctrica.

Eliminación de artefactos

El programa no realiza detección ni eliminación automática de:

- movimiento;
- parpadeos;

- actividad muscular;
- ruido producido por los electrodos.

Análisis frecuencial

No se implementa:

- FFT;
- PSD;
- espectrogramas;
- análisis de bandas EEG.

Clasificación

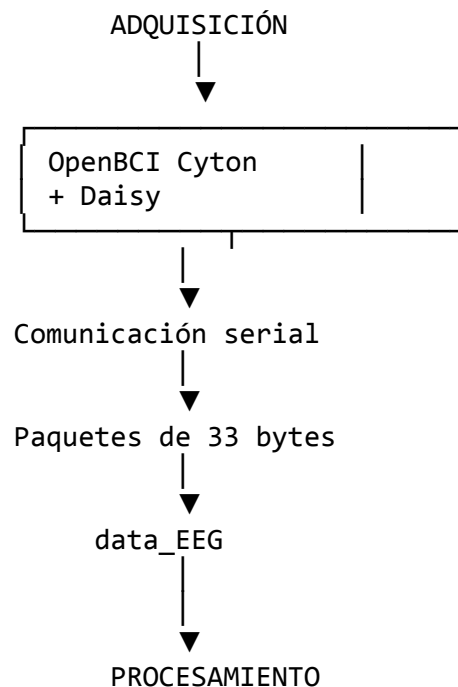
No existe actualmente ningún algoritmo de:

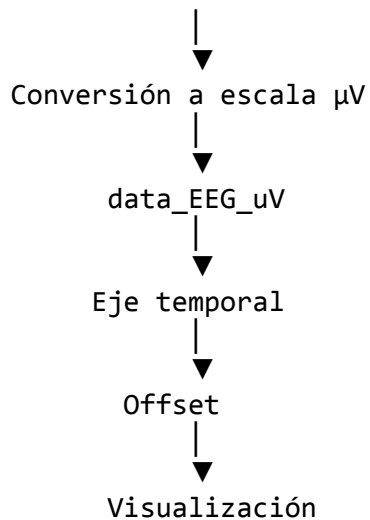
- clasificación;
- reconocimiento de patrones;
- detección automática de estados cerebrales.

Estas funcionalidades pueden incorporarse posteriormente como módulos independientes.

17. Relación entre adquisición y procesamiento

El proyecto puede dividirse en dos etapas principales:

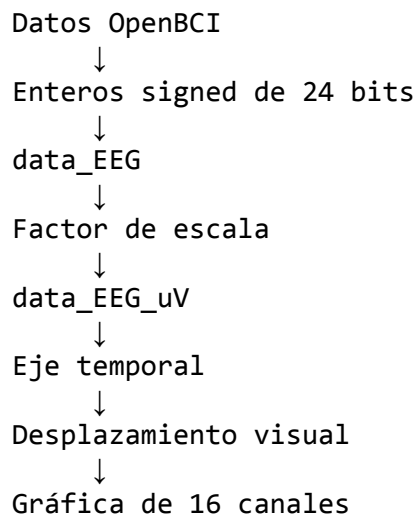




18. Resumen

El procesamiento implementado actualmente transforma los datos obtenidos durante la adquisición en una representación adecuada para su visualización.

El flujo es:



El programa conserva los datos originales en `data_EEG`, mientras que `data_EEG_uV` contiene los datos después de aplicar el factor de escala utilizado por la implementación.

La etapa de procesamiento actual es deliberadamente sencilla y constituye una base para futuras etapas de procesamiento digital de señales EEG.